CS-114

Programming Fundamentals

By

ALVINA ANJUM

Previous Lecture

Components of Computers

Hardware and Software

System Software and Application Software

Compiler

Interpreter

Today's Lecture

Computer Programming

Algorithm

Pseudo code

Pseudo code

Flow Chart

Flow Chart

Program Development Cycle

Programming Style and Documentation

Programming Errors

Why Programming?

- To learn how the computer works
- To develop problem solving skills
- To understand computer applications—games, business, scientific, engineering, network—which are all programs
- To build foundation for understanding specific areas of CS/CIS curricula—data structures, database systems, operating systems, network systems, etc.

Computer Programming

- A computer is a programmable machine. This means it can execute a programmed list of instructions and respond to new instructions that it is given.
- Computer Programming is the process of developing and implementing various sets of instructions to enable a computer to do a certain task.
- Programs are written to solve problems or perform tasks on a computer.

Computer Programming

- Programmers translate the solutions or tasks into a language the computer can understand.
- As we write programs, we must keep in mind that the computer will only do what we instruct it to do.
- Because of this, we must be very careful and thorough with our instructions.



First Computer Programmer: Ada Lovelace

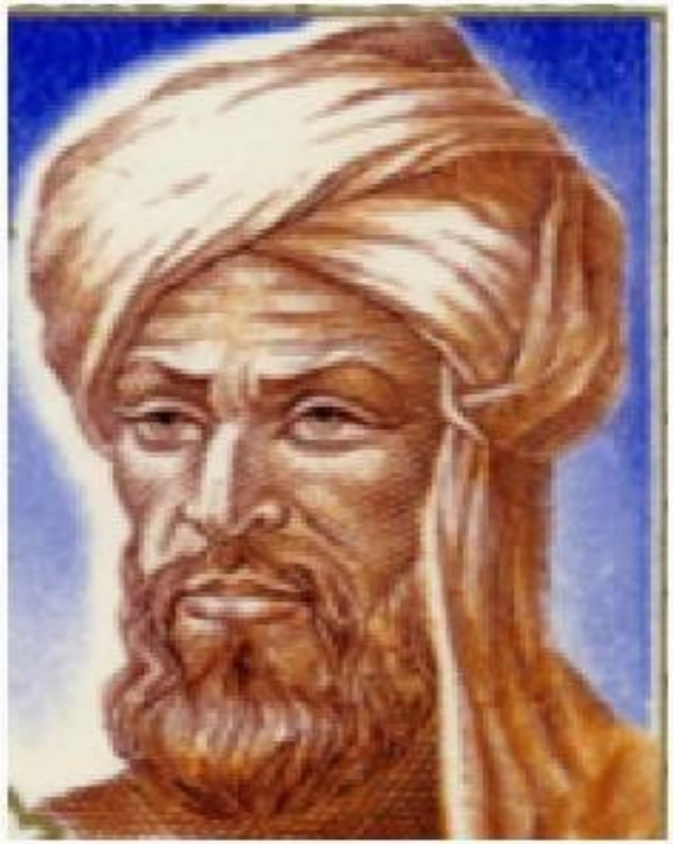


Ada Lovelace is the first person to develop an algorithm for a machine.

- In 1842-1843, Lovelace translated an article about Charles Babbage's proposed Analytic Engine. In her notes, she describes an algorithm that is cited as the first computer program, making her the first computer programmer.
- She also theorized that the computer could, one day, play music and chess.
- Ada, a U.S. Department of Defense computer language, is named in her honor.

Algorithm

- An **algorithm** is a list of instructions, procedures, or formulas used to solve a problem.
- The word derives from the name of the mathematician, Mohammed ibn-Musa al-Khwarizmi (**El-Harezmî**), (780 – 850).



Pseudocode

- Pseudocode is a computer programming language that resembles plain English that cannot be compiled or executed, but explains a resolution to a problem.

Source Code

- The source code consists of the programming statements that are created by a programmer with a text editor or a visual programming tool and then saved in a file.
- For example, a programmer using the C language types in a desired sequence of C language statements using a text editor and then saves them as a named file.
- This file is said to contain the source code.

Flowchart

- A flowchart is a formalized graphic representation of a logic sequence, work or manufacturing process, organization chart, or similar formalized structure.
- The purpose of a flow chart is to provide people with a common language or reference point when dealing with a project or process.
- Flowcharts use simple geometric symbols and arrows to define relationships.

Algorithms: Pseudo Code & Flow Chart

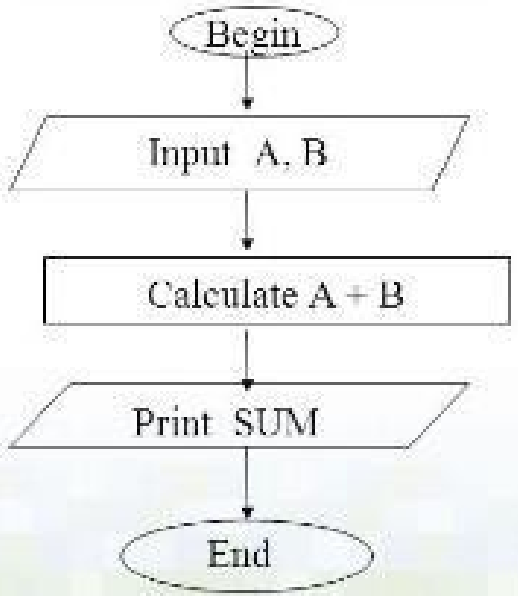
Example 2:

Write a flowchart that calculates and prints the SUM of two integers A and B.

Pseudo Code

- Begin
- Input A and B
- Calculate A + B
- Print result of SUM
- End

Flow Chart



Programming Languages

Machine Language Assembly Language High-Level Language

Machine language

- is a set of primitive instructions built into every computer.
- The instructions are in the form of binary code.
- Such programs are highly difficult to read and modify (for humans). For example, to add two numbers, you might write an instruction in binary like this:

```
1101101010011010
```

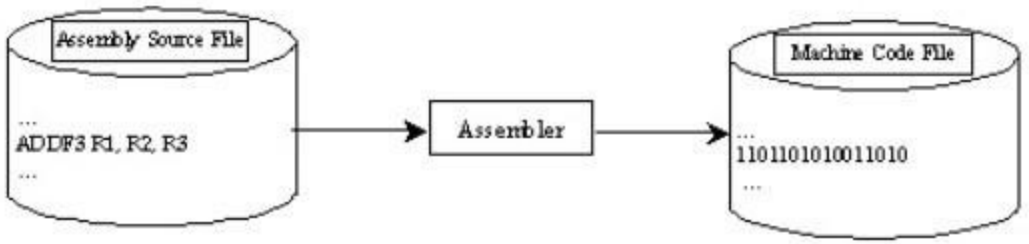
Programming Languages

Machine Language Assembly Language High-Level Language

Assembly languages

- were developed to make programming easy.
- A program called assembler is used to convert assembly language programs into machine code.
- For example, to add two numbers, you might write an instruction in assembly code like this:

ADDF3 R1, R2, R3



Programming Languages

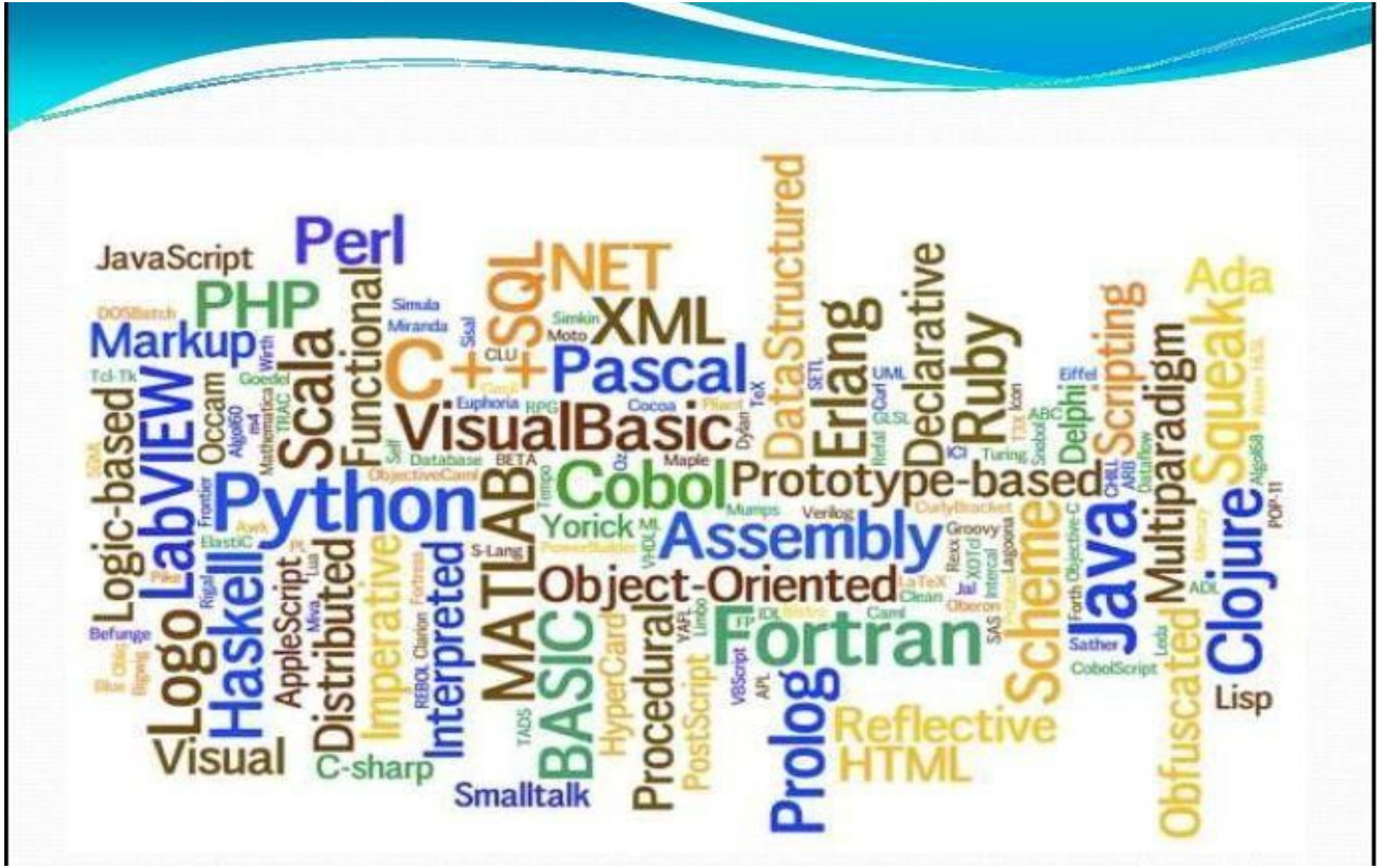
Machine Language Assembly Language High-Level Language

- **The high-level languages**
- are English-like and easy to learn and program.
- For example, the following is a high-level language statement that computes the area of a circle with radius 5:

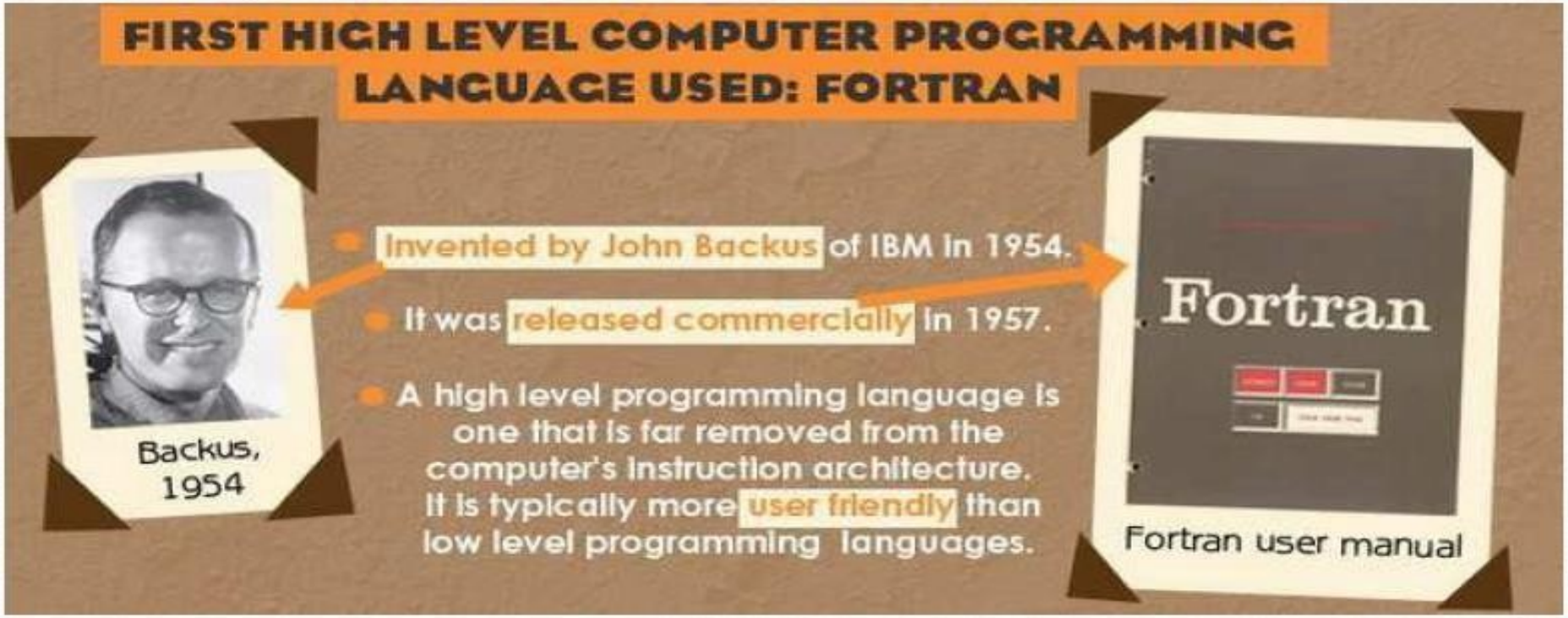
```
area = 5 * 5 * 3.1415;
```

Popular High-Level Languages

- COBOL (COmmon Business Oriented Language)
- FORTRAN (FORmula TRANslation)
- BASIC (Beginner All-purpose Symbolic Instructional Code)
- Pascal (named for Blaise Pascal)
- Ada (named for Ada Lovelace)
- C (whose developer designed B first)
- Visual Basic (Basic-like visual language developed by Microsoft)
- Delphi (Pascal-like visual language developed by Borland)
- C++ (an object-oriented language, based on C)


JavaScript
Perl
PHP
Markup
Scala
Functional
C++
SQL
NET
XML
Pascal
VisualBasic
DataStructured
Erlang
Declarative
Ruby
Ada
Squeak
Scripting
Multiparadigm
Logic-based
LabVIEW
Occam
Python
MATLAB
Cobol
Prototype-based
Yorick
Assembly
Object-Oriented
Haskell
Logo
AppleScript
Distributed
Imperative
Interpreted
BASIC
Procedural
HyperCard
PostScript
Fortran
Scheme
Java
Clojure
Lisp
Visual
C-sharp
Smalltalk
Prolog
Reflective
HTML
Obfuscated
Befunge
Simula
Miranda
Simkin
Moto
CLU
Euphoria
Cocoa
Database
Maple
Mumps
Verilog
Groovy
Delphi
Eiffel
ABC
Turing
Icon
UML
Sather
CobolScript
Forth
Objective-C
Lasso
ADL
Algol68
POP-11
Clean
Oberon
Caml
Limbo
Rexx
Lagoona
Intercal
Self
Oz
Dylan
TeX
Turing
ICI
Snobol
CHILL
ARB
Dataflow
Frontier
Awk
Lua
Mirva
Rigal
Pilot
Clarion
Fortress
REBOL
S-Lang
VHDL
ML
APL
SAS
Algol60
Mathematica
TRAC
Goedel
Wirth
Tcl-Tk
SQR
DOSBatch
Simula
Sisal
RPG
Pliant
TeX
SETL
Icon
Modula-3
Algol68
Modula-2
Ada95
TADS
YAFL
IDL
Basic
Curl
RLSL
Refal
Snobol
Database
BETA
Tempo
ObjectiveCaml
Fortress
Awk
Pilot
Rigal
Lua
Mirva
Clarion
REBOL
S-Lang
Frontier
Algol60
Mathematica
TRAC
Goedel
Wirth
Occam
Logic-based
Haskell
Logo
Visual
C-sharp
Smalltalk
Distributed
AppleScript
Imperative
Interpreted
BASIC
Procedural
HyperCard
PostScript
Fortran
Prolog
Reflective
HTML
Scheme
Java
Clojure
Lisp
Obfuscated
Squeak
Ada
Scripting
Multiparadigm
Delphi
Eiffel
Ruby
Declarative
Erlang
DataStructured
Pascal
XML
NET
SQL
C++
Functional
Scala
Markup
PHP
Perl
JavaScript
Python
VisualBasic
Cobol
Prototype-based
Assembly
Object-Oriented
Yorick
MATLAB
Befunge
Sather
CobolScript
Groovy
Verilog
Mumps
Lisp
Clojure
Java
Fortran
Scheme
Ruby
Erlang
Pascal
C++
SQL
NET
XML
Perl
PHP
Scala
Haskell
Logo
LabVIEW
Python
MATLAB
BASIC
Procedural
Prolog
Reflective
HTML
Visual
C-sharp
Smalltalk
Interpreted
Imperative
Distributed
AppleScript
Logic-based
Markup
JavaScript

First High Level Language



Design and Illustration
By Ellie Koning

Compiling Source Code

Source Program

- A program written in a high-level language

Compiler

- Software use to translate the source program into machine languageprogram

Object Code

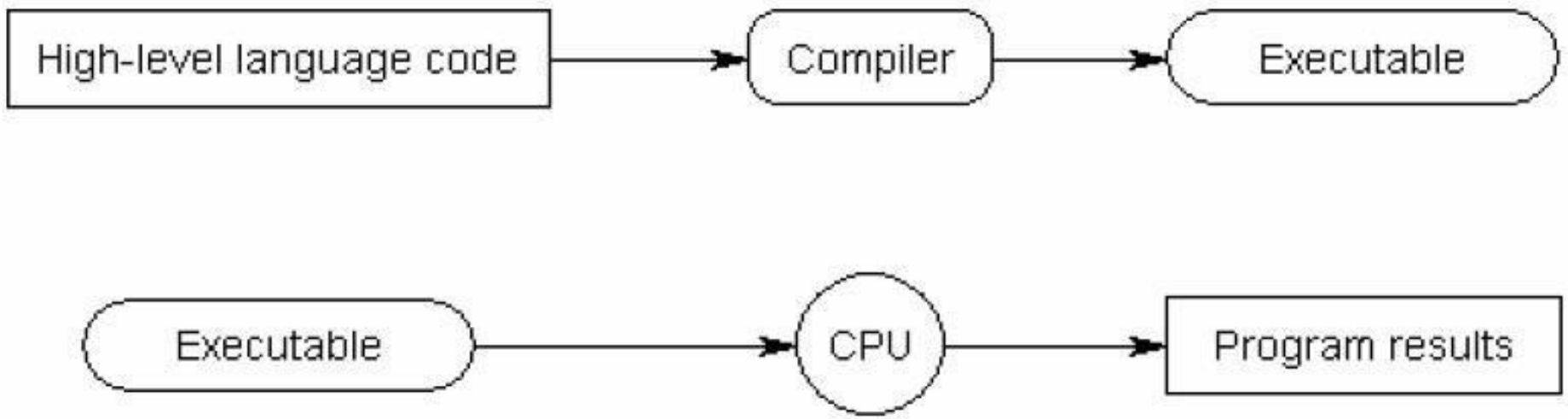
- Machine code resulting from compilation process

Executable Code

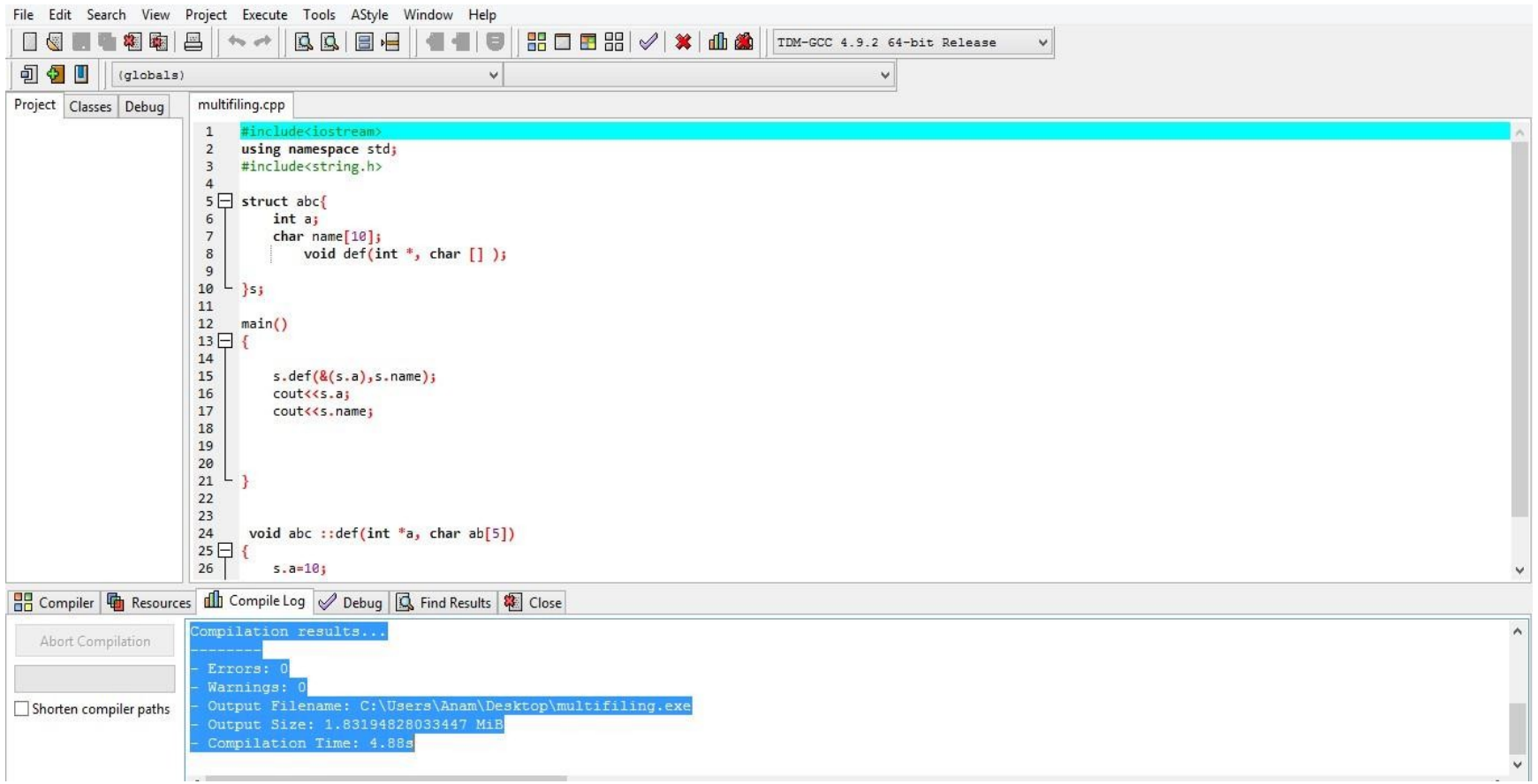
- Resulting from linking object code with other pieces of code to make the program ready for execution



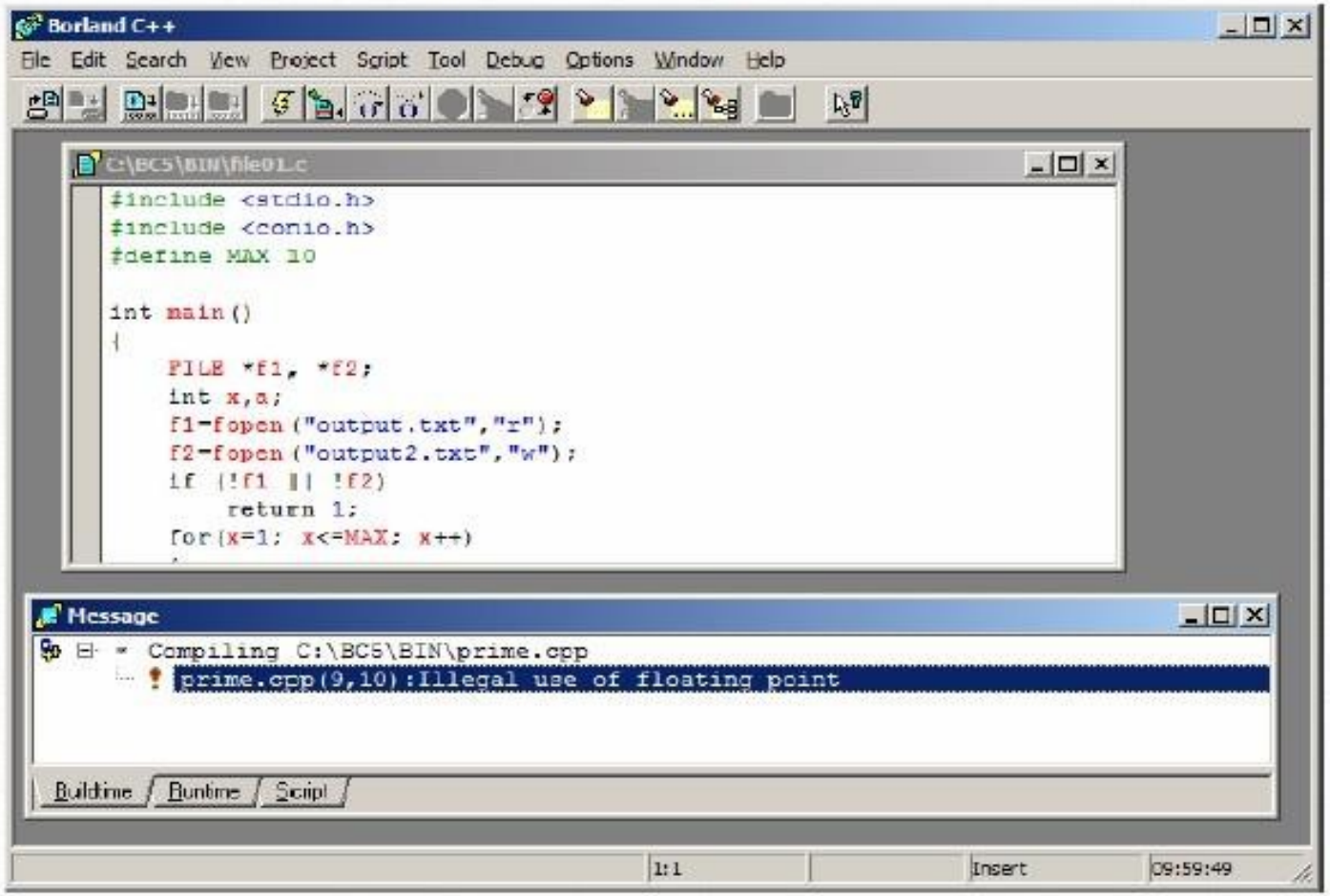
Compiler


High-level language code
Compiler
Executable
Executable
CPU
Program results

DEV C++ Compiler



Borland C++ Compiler



Integrated Development Environment (IDE)

An IDE or Integrated Development Environment is a software program that is designed to help programmers and developers build software.

Most IDEs include:

- a source code editor
- a compiler
- build automation tools
- a debugger

A SIMPLE C++ PROGRAM

```
#include <iostream>
int main()
{
  // Display Welcome to C++ to the console
  std::cout << "Welcome to C++!" << std::endl;
  return 0;
}
```

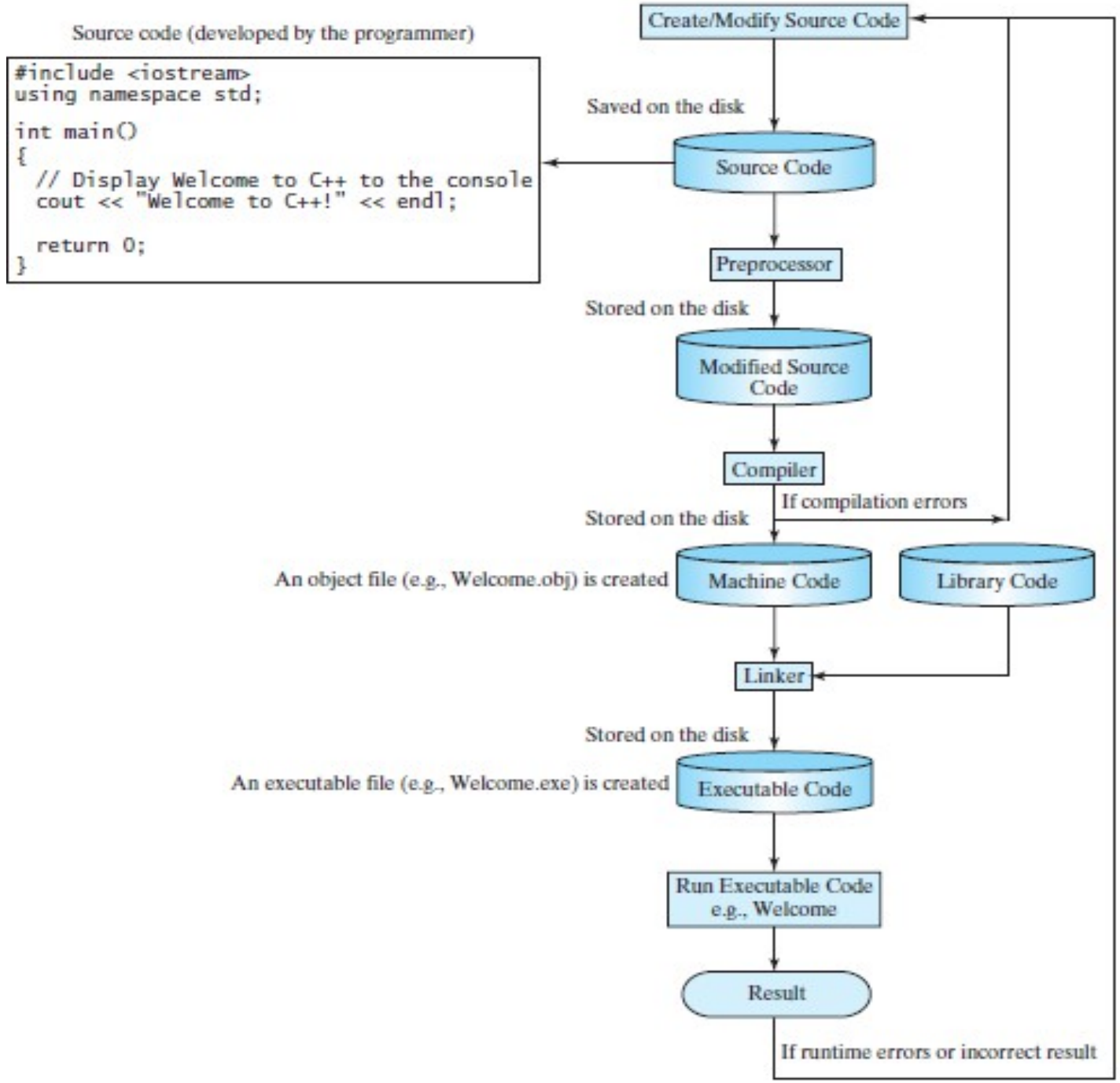
EXTENDING A SIMPLE C++ PROGRAM

```
#include <iostream>
int main()
{
  std::cout << "Welcome to C++!" << std::endl;
  std::cout << "Welcome to C++Builder!"
            << std::endl;
  std::cout << "Welcome to C++ Compiler!"
            << std::endl;
  return 0;
}
```

A Simple C++ Program

TABLE 1.2 Special Characters

Character	*Name*	*Description*
#	Pound sign	Used in `#include` to denote a preprocessor directive.
<>	Opening and closing angle brackets	Encloses a library name when used with `#include`.
()	Opening and closing parentheses	Used with functions such as `main()`.
{}	Opening and closing braces	Denotes a block to enclose statements.
//	Double slashes	Precedes a comment line.
<<	Stream insertion operator	Outputs to the console.
" "	Opening and closing quotation marks	Wraps a string (i.e., sequence of characters).
;	Semicolon	Marks the end of a statement.


Source code (developed by the programmer)
#include <iostream>
using namespace std;
int main()
{
// Display Welcome to C++ to the console
cout << "Welcome to C++!" << endl;
return 0;
}
Create/Modify Source Code
Saved on the disk
Source Code
Preprocessor
Stored on the disk
Modified Source Code
Compiler
If compilation errors
Stored on the disk
An object file (e.g., Welcome.obj) is created
Machine Code
Library Code
Linker
Stored on the disk
An executable file (e.g., Welcome.exe) is created
Executable Code
Run Executable Code e.g., Welcome
Result
If runtime errors or incorrect result

Programming Style and Documentation

Good programming style and proper documentation make a program easy to read and help programmers prevent errors

1) Appropriate Comments and Comment Styles
2) Proper Indentation and Spacing

```
cout << 3+4*4;          <-- Bad style
cout << 3 + 4 * 4;      <-- Good style
```

Task 01
Identify and fix the errors in the following code:

```
include <iostream>;
using namespace std;

int main
{
  // Display Welcome to C++ to the console
  cout << Welcome to C++! << endl;

  return 0;
}
```

.

Task 02
How do you denote a comment line and a comment paragraph?

Task 03
Reformat the following program according to the programming style and documentation guidelines

```
#include <iostream>
using namespace std;

int main()
{
cout << "2 + 3 = "<<2+3;
  return 0;
}
```

Programming Errors

Programming errors can be categorized into three types: syntax errors, runtime errors, and logic errors.

1) Syntax Errors

Errors that are detected by the compiler are called *syntax errors* or *compile errors*. Syntax errors result from errors in code construction, such as mistyping a keyword, omitting necessary punctuation, or using an opening brace without a corresponding closing brace.

```
#include <iostream>
using namespace std

int main()
{
  cout << "Programming is fun << endl;

  return 0;
}
```

Programming Errors

2) Runtime Errors

Runtime errors cause a program to terminate abnormally.

- ✓ They occur while an application is running if the environment detects an operation that is impossible to carry out. Input mistakes typically cause runtime errors.

- ✓ Another common source of runtime errors is division by zero.

```
#include<iostream>
using namespace std;
main()
{
    int a=10;
    int b=a/0;
```

3) Logic Errors

Logic errors occur when a program does not perform the way it was intended. Errors of this kind occur for many different reasons

Common Errors

- Common Error 1: Missing Braces

```
int main()
{

}
```

Type this closing brace right away to match the opening brace

- Common Error 2: Missing Semicolons

```
int main()
{
      cout << "Programming is fun!" << endl;
      cout << "Fundamentals First" << endl;
      cout << "Problem Driven" << endl
}
```

Missing a semicolon

Common Errors

- Common Error 3: Missing Quotation Marks

```
cout << "Problem Driven;
```

Missing a quotation mark

- Common Error 4: Misspelling Names

C++ is case-sensitive. Misspelling names is a common error made by new programmers. For example, the word main is misspelled as Main in the following code

```
int Main()
{
  cout << (10.5 + 2 * 3) / (45 - 3.5) << endl;
  return 0;
}
```